

Library Management System Project Report

1. Cover Page

- **Project Title:** Book Library Management System using Python
- **Prepared By:** SYED AKBER HUSSAIN
- **Date:** November 23, 2025

2. Introduction

The Book Library Management System (LMS) is a foundational software application designed to manage the key functions of a small-scale library. This project utilizes **Object-Oriented Programming (OOP) in Python** to efficiently track and manage the library's inventory of books. The primary goal is to provide a simple, robust, and interactive interface for library staff to perform daily operations such as adding, listing, searching, checking out, and returning books.

3. Problem Statement

Modern libraries, even small ones, require an efficient method to track their resources. Manually tracking book availability, check-out status, and inventory is prone to errors, time-consuming, and difficult to scale. The problem is to develop a **simple, data-centric system** that automates the inventory and circulation processes for books, providing accurate, real-time status of each item and improving the operational efficiency of the library staff.

4. Functional Requirements

These define what the system **must do**:

- **Book Management:**
 - The system must allow a user (Librarian) to **add new books** (Title, Author, ISBN).
 - The system must allow a user to **list all books** in the inventory.
 - The system must allow a user to **search for a book** by title (supporting partial and case-insensitive matching).
- **Circulation Management:**
 - The system must allow a user to **check out a book** by title, marking its status as "Checked Out."
 - The system must allow a user to **return a book** by title, marking its status as "Available."
- **Status Reporting:**
 - The system must display the current availability status (**Available** or **Checked Out**) for every book.

5. Non-functional Requirements

These define **how** the system performs:

- **Usability:** The system must use a **simple, text-based Command Line Interface (CLI)** that is easy to navigate.
- **Reliability:** The system should handle basic input errors gracefully (e.g., trying to check out a book that doesn't exist).
- **Maintainability:** The code must be well-structured, utilizing **Python classes** for clear separation of concerns (**Book** and **Library**).
- **Performance:** All core operations (add, list, search) must execute instantly, given the small scale of the inventory.

6. System Architecture

The system uses a **Two-Tier Architecture** (Presentation and Data/Application Logic combined) implemented entirely within a single Python script.

1. **Client/Presentation Layer:** The Command Line Interface (CLI) where the user interacts by calling the methods (e.g., `my_library.add_book(...)`).
2. **Application/Data Logic Layer:** This is handled by the Python classes:
 - The **Book** class defines the structure of the data object.
 - The **Library** class contains the core logic (methods) to manage the list of **Book** objects.

7. Design Diagrams

These diagrams visualize the structure and behavior of the system.

○ Use Case Diagram

- Description: Illustrates the interactions between the primary actor (Librarian/User) and the system's core functions.

\

○ Workflow Diagram

- Description: Shows the high-level flow of the main circulation tasks (e.g., Check Out process).

\

○ Sequence Diagram

- Description: Details the chronological order of interactions between the user and the objects (Library and Book) for a specific function, such as Check Out.

\

○ Class/Component Diagram

- **Description:** Shows the structure of the system, including the two main classes and their attributes/methods.
- **Classes:** **Book** and **Library**.
- **Relationship:** Library has a composition/aggregation relationship with Book (it contains a list of Book objects).

\

- **ER Diagram (if storage used)**

- **Description:** Since this version uses in-memory storage (a Python list), a traditional ER Diagram is not strictly necessary for persistence, but we can model the main entity.
- Entity: Book (Attributes: ISBN (PK), Title, Author, Status).

\

8. Design Decisions & Rationale

Design Decision	Rationale
OOP Approach	Encapsulates book data within the Book class and logic within the Library class, improving maintainability and modularity .
In-Memory Storage	Chosen for simplicity and fast development. Allows instant access to data without needing complex database setup or file I/O.
Case-Insensitive Search	Improves usability by allowing staff to search for titles without worrying about exact capitalization.
is_checked_out Boolean Flag	Simple and efficient way to track circulation status directly within the Book object.

9. Implementation Details

The system is implemented using **Python 3**.

- **File Structure:** A single file (**library_management.py**).
- **Data Structure:** A Python **list** (**self.books**) is used within the **Library** class to hold all **Book** objects.
- **Core Methods:** The search functionality uses a **linear search** (iterating through the list) which is adequate for a small-scale system.

10. Screenshots / Results

Screenshot 1: Adding Books

```
Book 'The Great Gatsby' added successfully.  
  
Book '1984' added successfully.  
  
Book 'The Great Gatsby' added successfully.  
  
Book '1984' added successfully.  
  
Book 'Pride and Prejudice' added successfully.  
  
Book '1984' added successfully.  
  
Book 'Pride and Prejudice' added successfully.  
  
Book 'Pride and Prejudice' added successfully.
```

Screenshot 2: Listing All Books

```
--- Current Library Collection ---  
[1] Title: The Great Gatsby, Author: F. Scott Fitzgerald, ISBN: 978-0743273565, Status: Available  
[1] Title: The Great Gatsby, Author: F. Scott Fitzgerald, ISBN: 978-0743273565, Status: Available  
[2] Title: 1984, Author: George Orwell, ISBN: 978-0451524935, Status: Available  
[3] Title: Pride and Prejudice, Author: Jane Austen, ISBN: 978-0141439518, Status: Available
```

Screenshot 3: Search Functionality

```
--- Found 1 Book(s) ---  
Title: The Great Gatsby, Author: F. Scott Fitzgerald, ISBN: 978-0743273565, Status: Available  
Title: The Great Gatsby, Author: F. Scott Fitzgerald, ISBN: 978-0743273565, Status: Available  
-----  
  
No book found with title matching 'problem solving'.
```

11. Testing Approach

The system was tested using a **Unit and Integration Testing** approach:

- **Unit Tests:**
 - Tested the **Book** object initialization and `__str__` method for correct output.
 - Tested individual **Library** methods (e.g., calling `add_book()` and verifying the list size increased).
- **Integration Tests (Scenario Testing):**
 - **Check Out Scenario:** Added a book, checked it out, and verified its status changed to "Checked Out."
 - **Return Scenario:** Checked out a book, returned it, and verified its status reverted to "Available."
 - **Edge Cases:** Tested attempting to check out an already checked-out book and attempting to return an available book.

12. Challenges Faced

- **Data Persistence:** The initial challenge was realizing that the data was lost every time the program terminated. The solution was to define **Future Enhancements** to handle file-based storage, keeping the current version simple.
- **Search Logic:** Implementing a robust search that handled **partial and case-insensitive matching** required careful use of Python's `.lower()` method.

13. Learnings & Key Takeaways

- **Mastery of OOP:** Gained practical experience in applying core OOP principles (Encapsulation, Classes, Objects) in Python.
- **Modeling Real-World Systems:** Learned how to translate real-world library operations (circulation, inventory) into software logic and data structures.
- **Importance of `__str__`:** Recognized the utility of special methods like `__str__` for creating human-readable output for complex objects.

14. Future Enhancements

- **Data Persistence:** Implement saving the book list to a file so data is not lost on exit.
- **User Interface:** Upgrade from a CLI to a graphical user interface (GUI) using libraries like **Tkinter** or **PyQt**.
- **Member Management:** Introduce a separate **Member** class to track users and link book check-outs to specific members.
- **ISBN Validation:** Add input validation to ensure the ISBN is in a correct format upon entry.

15. References

- *Python 3 Documentation* (docs.python.org)
- *Online Tutorials for Python OOP Concepts*

- *Specific textbook used for Python programming*